Campus Parking Occupancy Prediction and an Intelligent Parking Assistant Using Leakage-Safe Tree Ensembles and SHAP Explanations

Tanya Agrahari
M.Tech (AI & ML), CSE513 Machine Learning
Department of Machine Learning and Artificial Intelligence
Lovely Professional University, Phagwara, Punjab, India
Reg. No. 12602327 · Section K3P26ML

***Abstract*—Drivers waste time and fuel searching for parking. This project builds a machine-learning system that predicts how full each parking zone will be, recommends where to park, and shows administrators how parking demand behaves. Real sensor data from 28 car parks recorded over 77 days is used as a stand-in for campus parking zones. The raw readings are cleaned and arranged into 30-minute time slots, and two forecasting tasks are solved: predicting occupancy one hour ahead and predicting it for the whole of the next day. Five machine-learning models (ridge regression, support vector regression, random forest, gradient boosting and XGBoost) and a combined "stacked" model are compared against three simple rule-based forecasts. All models are tested only on dates they never saw during training. The chosen random forest model is off by 0.0217 occupancy rate on average one hour ahead (about 26.1 vehicles), which is 60.6% better than the best simple rule, and 0.0479 (about 61.4 vehicles) for the next day, 11.3% better than the best simple rule. SHAP explanations show which inputs drive each prediction, and an error analysis shows that most mistakes happen on weekends, when car parks are nearly full, and on days with unusual sensor readings. The project also reports, and corrects, a mistake in how the final model was first chosen. A web application built with Streamlit puts the forecasts in the hands of drivers and administrators.**

***Index Terms*—parking occupancy prediction, smart campus, random forest, XGBoost, SHAP, time-series validation, Isolation Forest, Streamlit.**

I. INTRODUCTION

On a university campus, parking demand rises and falls with the timetable, so certain zones fill up at predictable hours. If drivers knew in advance which zones would still have free spaces, they could drive straight there instead of circling, and administrators could plan capacity better. The project brief asks for a system that predicts zone-level availability from past occupancy and calendar information, recommends likely available areas, and is deployed with an admin dashboard.

The main contributions of this work are:

- a complete pipeline that turns a messy, irregular sensor log into a clean table of occupancy per zone, per day and per 30-minute slot, together with an Isolation Forest that flags suspicious sensor days for the administrator instead of silently deleting them;
- a fair comparison of five tuned machine-learning models, a stacked model and three simple rule-based forecasts on two forecast horizons, with confidence ranges and significance tests to check that differences are real and not due to luck;
- SHAP explanations and a detailed error analysis that show *why* the models predict what they do and *where* they go wrong;
- a deployed Streamlit web app with a driver view, a zone forecast view and an admin dashboard.

The report also explains openly how the first rule for choosing the final model went wrong and how it was fixed.

II. RELATED WORK

Caicedo *et al.* [1] predicted real-time parking availability for reservation systems by simulating cars arriving and leaving. Rajabioun and Ioannou [2] used statistical time-series models on San Francisco and Los Angeles data and recommended the zone most likely to have a free space at arrival time. Vlahogianni *et al.* [3] used neural networks to predict short-term street parking occupancy in Santander. Stolfi *et al.* [4], [5] introduced the Birmingham data set used here and compared several prediction methods on it, and Camero *et al.* [6] later applied deep recurrent neural networks to the same data. Xiao *et al.* [7] give a survey of the field. Compared with these works, this project focuses on fair testing, comparison against strong simple forecasts, explaining the model with SHAP [10], [11] and a working tool for both drivers and administrators. The models used are random forests [8], gradient-boosted trees [9], [12] and support vector regression, all implemented with the scikit-learn library [13].

III. DATA AND EXPLORATORY ANALYSIS

A. Data Source

No campus parking log was publicly available, so the public *Parking Birmingham* data set [14] was used. It records how many cars were parked in each of 30 car parks roughly every 30 minutes from 08:00 to 16:30, between 4 October and 19 December 2016. Car park sizes range from 220 to 4,675 spaces. Like campus zones, these car parks have fixed capacities and clear daily and weekly patterns, so they are a reasonable stand-in. The code only needs four columns (zone, capacity, number of cars, time), so a real campus log can be used later without changing the code.

B. Cleaning the Data

Raw sensor data is messy. Table I lists each cleaning step and how many records it affected. The *occupancy rate* is the number of parked cars divided by capacity, so 0 means empty and 1 means full. 373 readings showed more cars than spaces, which is impossible, so they were capped at 1. Each reading was moved to the nearest 30-minute slot (18 slots per day), and two readings in the same slot were averaged. Car parks with fewer than 30 days of data were removed. Missing slots (9.35% of the table) were left empty rather than filled with guessed values, so the models never learn from invented data.

TABLE I
DATA CLEANING SUMMARY

Step	Value
Raw readings / car parks	35,717 / 30
Exact duplicates removed	216
Car parks removed (< 30 days)	2
Readings above capacity (capped)	373
Readings outside 08:00–16:30	21
Same-slot readings averaged	52
Clean readings / car parks	35,181 / 28
Table cells (zone × day × slot)	38,808
Missing cells	9.35%
Zone-days flagged as unusual	40

C. What the Data Shows

The average occupancy rate is 49.7%. Car parks are busiest at 14:00 (61.6% full on average); Thu is the busiest day and Sun the quietest (Fig. 1). The assistant labels each slot as *Available* (below 70% full), *Limited* (70–90%) or *Full* (90% or more). In the data, 72.9% of slots are Available, 18.5% Limited and only 8.6% Full, so the classes are unbalanced.

Occupancy changes slowly and repeats itself. The correlation between a slot and the slot just before it is 0.985 (two slots before: 0.947). With the same slot yesterday it is 0.797, and with the same slot last week it is 0.852. (A correlation of 1 would mean a perfect match.) This is why recent and past values were used as model inputs.

Demand also grows as Christmas approaches: the average rate is 47.5% in the training period, 53.5% in the validation period and 53.3% in the test period. This shift turned out to matter when choosing the final model (Section V-B).

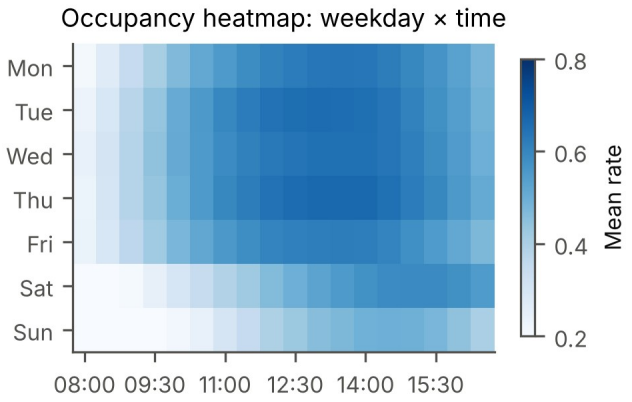


Fig. 1. Average occupancy by weekday and time of day across all car parks. Darker means fuller.

Grouping similar car parks. Each car park's average weekday pattern was grouped using K-means clustering (an algorithm that puts similar items into the same group), trying 2 to 6 groups. The silhouette score, which measures how well-separated the groups are, was best with 2 groups (0.597). This gives a quieter group of 13 car parks (average 32.9% full) and a busier group of 15 (average 70.5%). A second method, hierarchical clustering, partly agrees (adjusted Rand index 0.492, where 1 means identical groups).

Finding unusual days. An Isolation Forest [15] is an algorithm that finds unusual records by checking how easily each one can be separated from the rest. It looked at every car park's day (average, spread, maximum, repeated identical readings and difference from normal) and flagged 40 unusual zone-days, such as a stuck sensor or a special event. These days are shown to the administrator rather than deleted, because a real event day is useful information.

IV. METHODOLOGY

A. Problem Definition

Let $r_{z,d,s}$ be the occupancy rate (0 to 1) of zone z on day d in time slot s. Two prediction tasks are solved:

$$\hat{r}_{z,d,s+2} = f_{\text{short}}(x_{z,d,\le s}) \quad (1)$$

$$\hat{r}_{z,d,s} = f_{\text{day}}(x_{z,\le d-1}) \quad (2)$$

Task (1) predicts one hour (two slots) ahead, using today's readings up to now. Task (2) predicts every slot of tomorrow, using only data up to today. Each predicted rate is turned into a label (Available / Limited / Full) and multiplied by capacity to give the expected number of free spaces. Fig. 2 gives an overview of the whole system, from the raw sensor data to the app.

B. Model Inputs (Features)

The inputs given to the models are listed in Table II. A key rule was followed: **every input uses only information that would really be known at the moment of prediction.** Using future information by mistake, called *data leakage*, makes a model look much better in testing than it is in real life. One

input, the *zone profile*, is the typical occupancy for that car park at that time on a weekday or weekend. It is learned only from training data, inside the model pipeline, so it never "sees" the test answers. Automatic tests check these rules.

TABLE II
MODEL INPUTS (FEATURES)

Group	Features	Task
Calendar	time slot, hour, weekday, weekend flag, days to Christmas	both
Past days	same slot yesterday and last week, 7-day average, yesterday's average	both
Today so far	latest rate, two earlier rates, latest change, today's average so far	1 h
Car park	capacity, car park identity, typical profile	both

C. Models Compared

A model is only useful if it beats simple common-sense rules, so three *baselines* were used: "same as the last reading" (or "same as yesterday" for next-day forecasts), "same as this time last week", and "the usual value for this car park and time". Five machine-learning models were then trained:

- **Ridge regression:** a straight-line (linear) model, the simplest learner.
- **Support vector regression (SVR) with an RBF kernel:** a model that can fit curved patterns using the kernel trick.
- **Random forest [8]:** many decision trees, each trained on a random part of the data, whose answers are averaged.
- **Histogram gradient boosting and XGBoost [9]:** trees built one after another, each new tree correcting the errors of the previous ones.

A *stacked ensemble* [16] combined the five models by learning how much weight to give each. In addition, two extra models, trained with quantile (pinball) loss [17], give a likely range (an 80% prediction interval) for each forecast, and a separate classifier predicts the Available / Limited / Full label directly.

D. How the Models Were Tested

The data was split by date, never shuffled, so the models were always tested on days that came after the days they learned from, just like real forecasting: training 4 Oct–20 Nov, validation 21 Nov–4 Dec (12 days with data, 2 of them weekend days) and test 5–19 Dec (15 days, 4 weekend days). The settings of each model (hyper-parameters) were tuned on the training period with *time-series cross-validation*: the model is repeatedly trained on earlier weeks and checked on the following weeks, four times. Models were then retrained on training plus validation data and checked **once** on the test period. The whole training run took 22 minutes.

The main error measure is the *mean absolute error* (MAE): the average size of the mistake. For example, an MAE of 0.05 means the forecast is off by 5 percentage points of occupancy on average. To know how much to trust each MAE, a 95% confidence range was computed with a *bootstrap*: the test days are resampled many times and the MAE recomputed each time. Whole days were resampled together, because errors on the same day are related. The *Wilcoxon test* checks whether one model's errors are truly smaller than another's or the difference is just luck, and *McNemar's test* does the same for two labelling methods.

How the final model was chosen. The original plan was to deploy the model with the lowest error on the validation period. That rule chose SVR for both tasks, but SVR then did poorly on the test period (Section V-B). The rule was changed to use the average of the cross-validation error and the validation error, which chose random forest for both tasks. Because this change was made *after* the test results were seen, the result of the chosen model should be read with that in mind; to keep things fair, the test results of **every** model are reported.

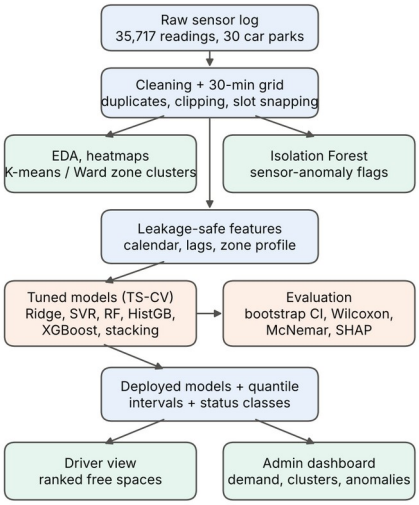


Fig. 2. System overview, from the raw sensor data to the driver and admin views.

V. RESULTS

A. Forecast Accuracy

Tables III and IV show the test results. MAE and RMSE are in occupancy rate (lower is better). RMSE is like MAE but punishes large mistakes more. R^2 shows how much of the variation the model explains (1 is perfect). "Veh." gives the MAE in number of vehicles, and F1 measures how well the Available / Limited / Full label is predicted (1 is perfect).

TABLE III
TEST RESULTS, 1-HOUR-AHEAD (6,222 PREDICTIONS)

Model	MAE	RMSE	R^2	veh.	F1
XGBoost	0.0203	0.0292	0.988	24.9	0.920
HistGB	0.0206	0.0294	0.987	25.2	0.917
Random Forest	**0.0217**	**0.0318**	**0.985**	**26.1**	**0.916**
Stacked ensemble	0.0248	0.0347	0.983	31.9	0.908
SVR (RBF)	0.0283	0.0392	0.978	37.2	0.900
Ridge	0.0320	0.0453	0.970	40.6	0.886
Seasonal naive (last week)	0.0552	0.0833	0.899	66.4	0.793
Persistence (last value)	0.0713	0.1012	0.852	97.8	0.770
Historical profile	0.0836	0.1089	0.828	112.3	0.533

F1 = macro-F1 of the Available / Limited / Full label. Bold = model used in the app.

For one-hour-ahead forecasts, all five machine-learning models beat all three simple rules by a wide margin. The chosen random forest has an MAE of 0.0217 (95% range [0.019, 0.024]), compared with 0.0552 for the best simple rule, "same as last week" (range [0.042, 0.071]). That is 60.6% smaller, or 26.1 vehicles instead of 66.4. XGBoost (0.0203) and gradient boosting (0.0206) are very slightly better, but the three tree models differ by only about 0.001 and their ranges overlap, so they perform about equally well.

TABLE IV
TEST RESULTS, NEXT-DAY (7,102 PREDICTIONS)

Model	MAE	RMSE	R^2	veh.	F1
Random Forest	**0.0479**	**0.0721**	**0.930**	**61.4**	**0.808**
XGBoost	0.0504	0.0759	0.923	64.4	0.789
HistGB	0.0522	0.0787	0.917	66.8	0.801
Seasonal naive (last week)	0.0540	0.0825	0.909	65.6	0.794
Stacked ensemble	0.0564	0.0822	0.910	74.7	0.762
Ridge	0.0622	0.0859	0.901	78.7	0.744
SVR (RBF)	0.0652	0.0917	0.887	88.3	0.740
Historical profile	0.0824	0.1080	0.844	111.1	0.522
Persistence (yesterday)	0.0906	0.1516	0.692	106.3	0.704

Predicting a whole day ahead is much harder. Random forest has the lowest MAE, 0.0479 (range [0.037, 0.060]), which is 11.3% better than "same as last week" at 0.0540 (range [0.041, 0.069]). The Wilcoxon test says the difference is very unlikely to be luck (p = 1.5e-21), but the confidence ranges overlap, so the improvement is real but modest. Ridge, SVR and the stacked model (0.0564) are actually *worse* than "same as last week". In other words, knowing what happened at this time last week is already a strong forecast. Fig. 3 shows this comparison.

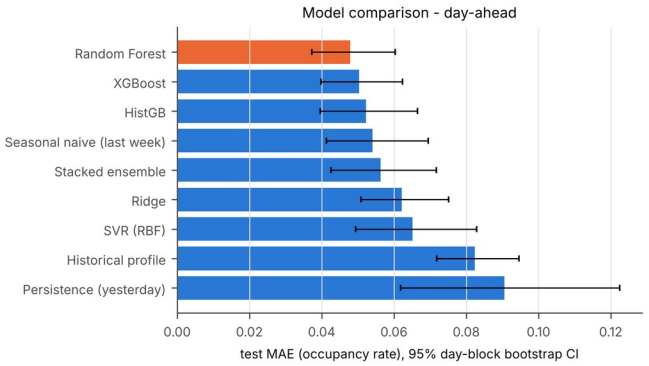


Fig. 3. Next-day test MAE of every model with 95% confidence ranges. Orange = model used in the app.

B. A Lesson in Choosing the Final Model

Table V explains why the first rule went wrong. SVR had the lowest validation error in both tasks, yet it had the highest cross-validation error of all five models and a poor test result. Looking closer, SVR's mistakes are concentrated on weekends: for next-day forecasts its test MAE is 0.110 on weekend days but 0.050 on weekdays (random forest: 0.074 and 0.039). The validation period had only 2 weekend days out of 12, while the test period had 4 out of 15, so this weakness was hidden during validation. Had SVR been deployed, its next-day forecasts (MAE 0.0652) would have been clearly worse than simply using last week's value (Wilcoxon p = 1.7e-62). The stacked model had the same problem, because it gave SVR the largest weight (0.76 for 1 hour and 0.73 for next day). **The lesson: a short validation period can be misleading. Combining it with cross-validation over the longer training period gives a more reliable choice.**

TABLE V
CHOOSING THE MODEL: CROSS-VALIDATION, VALIDATION AND TEST MAE (NEXT-DAY)

Model	CV	Valid.	Rule	Test
Ridge	0.0767	0.0680	0.0723	0.0622
SVR (RBF)	0.0801	0.0463	0.0632	0.0652
Random Forest	**0.0590**	**0.0509**	**0.0550**	**0.0479**
HistGB	0.0603	0.0555	0.0579	0.0522
XGBoost	0.0598	0.0573	0.0585	0.0504

Rule = average of CV and validation MAE (lower is better). For 1-hour-ahead the same rule also picks random forest (0.0247 vs. XGBoost 0.0252).

C. Available / Limited / Full Labels

For the parking assistant, the label matters more than the exact rate. Table VI compares two ways of getting it: (a) taking the random forest's predicted rate and applying the 70% and 90% cut-offs, and (b) a separate classifier trained to predict the label directly. Method (a) is more accurate overall in both tasks; McNemar's test confirms this difference is not luck (p = 0.008 for 1 hour and 2.4e-4 for next day). However, for next-day forecasts, method (b) catches more of the slots that are really Full (recall 0.901 vs. 0.729), at the cost of more false "Full" warnings (precision 0.676 vs. 0.783). Since sending a driver to a full car park is worse than a false warning, method (b) is a sensible choice for next-day "Full" warnings.

TABLE VI
LABEL PREDICTION ON THE TEST PERIOD

Task / method	Acc.	Macro-F1	Full recall
1 h, rate + cut-offs	0.941	0.916	0.868
1 h, direct classifier	0.934	0.912	0.911
Next day, rate + cut-offs	0.881	0.808	0.729
Next day, direct classifier	0.868	0.803	0.901

Recall = share of truly Full slots that were caught. Precision = share of Full warnings that were correct.

D. Likely Ranges (Prediction Intervals)

The 80% likely ranges should contain the true value 80% of the time. In practice they contained it 69.5% of the time for 1-hour forecasts (average width 0.063) and 64.0% for next-day forecasts (width 0.105). So the ranges are too narrow and should be widened in future work, for example with a method called conformal calibration. The app therefore calls them a "likely range" rather than a guarantee.

VI. EXPLAINING THE MODEL WITH SHAP

Machine-learning models can feel like black boxes. SHAP [10] opens the box by splitting each prediction into contributions from each input: prediction = average prediction + sum of the contributions. For tree models, these contributions

can be computed exactly [11]; as a check, they added up to the model's output with a largest error of only 2.0e-06 over all test predictions. SHAP was applied to the XGBoost model because it is fast for this kind of model; on the much larger random forest it took several minutes for just a few hundred predictions. To check that the random forest relies on the same inputs, *permutation importance* was also used: each input is shuffled in turn, and the more the error grows, the more important that input is. The 28 car-park identity columns are counted together as one input called "zone".

For 1-hour forecasts, the latest reading is by far the most important input (average SHAP size 0.1853), followed by the zone profile (0.0322) and the latest change (0.0287). Permutation importance on the random forest agrees: shuffling the latest reading increases its MAE by 0.1906. For next-day forecasts, the car park's typical profile matters most (0.1542), then last week's value (0.0465) and yesterday's value (0.0264), as Fig. 4 shows; higher values push the forecast up, as one would expect. For the random forest, shuffling the car-park identity (which also scrambles the typical profile) increases MAE by 0.0816, more than any other input. Fig. 5 explains one real prediction: for car park BHMBCCTHL01 on 5 December at 11:00, the model predicted 0.953 (actual: 1.000). Starting from the average of 0.519, the latest reading alone pushed the forecast up by 0.258. These patterns match common sense, which increases trust in the model.

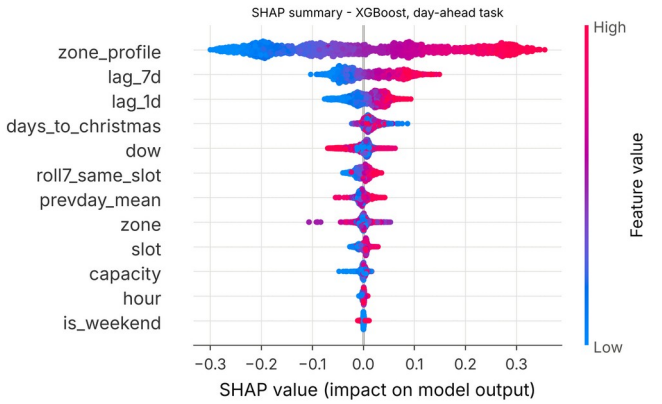


Fig. 4. SHAP summary for next-day forecasts (XGBoost). Each dot is one prediction; red = high input value, blue = low. Dots to the right push the forecast up.

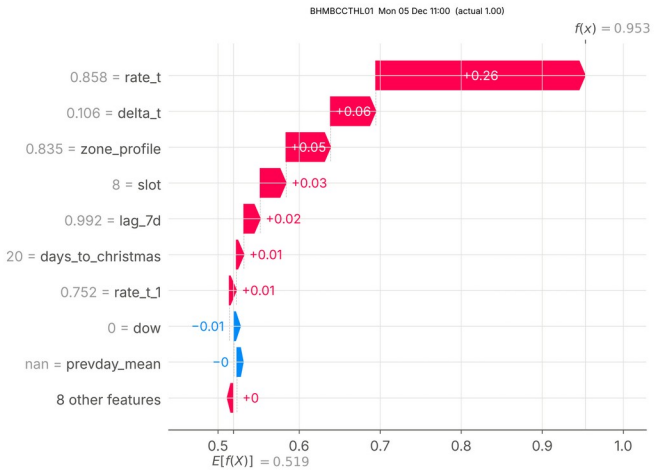


Fig. 5. SHAP explanation of one 1-hour-ahead prediction: how each input moves the forecast away from the average.

VII. ERROR ANALYSIS

An average error hides where a model fails, so the errors of the chosen model were split into groups. Tables VII and VIII group them by how full the car park really was. Errors grow as car parks fill up, and near capacity the model tends to predict *too low* (average bias -0.0488 for next-day forecasts when 90% or more full). This is partly because a rate cannot go above 1, so the model is pulled towards average values. For next-day forecasts, 21 slots that were really Full were predicted as Available (1 for 1-hour forecasts).

TABLE VII
CHOSEN MODEL'S MAE BY TRUE OCCUPANCY LEVEL (1-HOUR-AHEAD)

Level	Count	MAE	Bias
<30%	1290	0.0169	+0.0034
30-50%	1282	0.0223	-0.0013
50-70%	1407	0.0240	+0.0006
70-90%	1468	0.0228	-0.0036
≥90%	775	0.0229	-0.0161

TABLE VIII
CHOSEN MODEL'S MAE BY TRUE OCCUPANCY LEVEL (NEXT-DAY)

Level	Count	MAE	Bias
<30%	1799	0.0286	+0.0127
30-50%	1544	0.0493	-0.0005
50-70%	1478	0.0553	-0.0059
70-90%	1486	0.0580	-0.0063
≥90%	795	0.0565	-0.0488

Bias = average of (predicted − actual). Negative = the model predicts too low.

Over the day (Fig. 6), next-day error grows from 0.0319 at 08:00 to 0.0540 at 16:30, because afternoon demand is less predictable from the day before. Weekends are hardest: next-day MAE is 0.0754 on Saturdays and 0.0722 on Sundays, compared with 0.0280 on Tuesdays. On the days flagged as unusual by the Isolation Forest, next-day MAE is 0.0974 (250 predictions) compared with 0.0461 on normal days, which shows the flags really do catch unusual days. The single largest

errors happened on 6 December in car park BHMBCCSNH01, a flagged day: it was already full at 08:00, while the forecast was 0.35. The hardest car parks to predict were BHMBCCPST01 (0.085), BHMNCPNST01 (0.072), BHMBCCSNH01 (0.070), and the easiest was Others-CCCPS202 (0.018). Fig. 7 shows how often each label was predicted correctly.

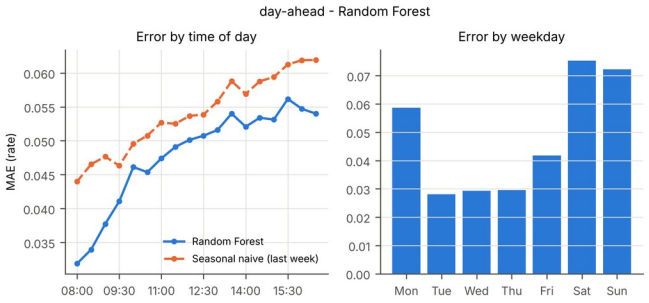


Fig. 6. Next-day MAE by time of day (random forest vs. the "same as last week" rule) and by weekday.

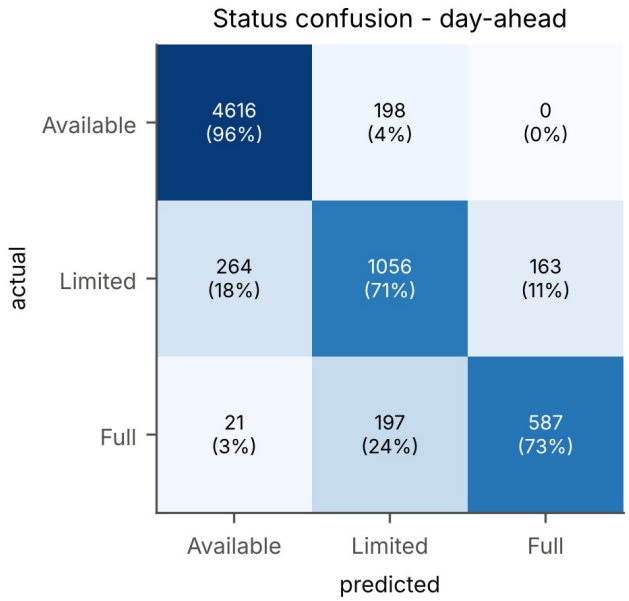


Fig. 7. Next-day label results for the chosen model: rows are the true label, columns the predicted label (counts and row percentages).

VIII. DEPLOYMENT

The system is delivered as a Streamlit web app with four pages. *Find parking* lets a driver choose an arrival day and time and ranks car parks by expected free spaces, with the label, likely range and capacity (Fig. 8). A slider hides car parks with too few free spaces, and a replay switch reveals what actually happened. *Zone forecast* draws the next-day and 1-hour forecasts for one car park against the real values. The *Admin dashboard* shows the busy-time heatmaps, the car-park groups, the list of unusual days and the model results. *How the model works* shows the SHAP charts.

Because the data ends in 2016, the app works in "replay" mode over the test period. The final random forest models are large (100–170 MB each) because their trees are deep, which is more than GitHub allows for a single file. So the models produce their forecasts for the whole test period once in advance (batch scoring), and the app reads these saved forecasts; they match the models' own output to within rounding (5×10^{-6}). The code, data and results are available at https://github.com/TanyaAgrahari9/Parking-Intelligence, and the app runs on Streamlit Community Cloud at https://tanya-parking-intelligence.streamlit.app/.

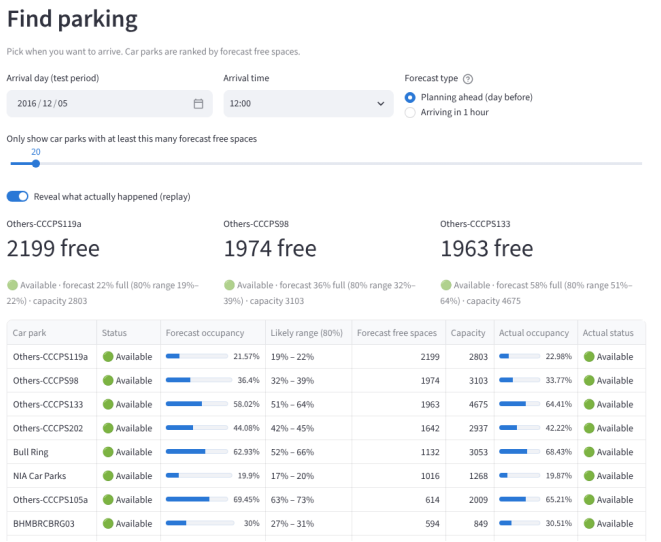


Fig. 8. The Find parking page: car parks ranked by expected free spaces for a chosen time, with the real outcome revealed.

IX. LIMITATIONS

- **Stand-in data.** The Birmingham car parks are in a city centre, not on a campus, and the data has no timetable information. It also counts only cars, so the current models cannot forecast two-wheeler parking. The results show the method works on real parking data, but not how accurate it would be on a specific campus.
- **Short time span.** The data covers only eleven weeks before Christmas, when demand keeps rising. The test weeks are the busiest shopping weeks, and the "days to Christmas" values in the test period are outside the range seen in training, which tree models cannot extend beyond.
- **Model choice changed after testing.** The rule for choosing the final model was changed after the test results were seen. All models' results are reported so readers can judge for themselves.
- **Significance tests may be too confident.** The Wilcoxon and McNemar tests assume each prediction is independent, but predictions close in time are related. The day-based confidence ranges are the more reliable guide.
- **Likely ranges too narrow.** The 80% ranges cover the truth less often than they should and need calibration.
- **No camera module.** The optional extension that detects free spaces from camera images was not built; occupancy comes from car-park counters.

X. CONCLUSION AND FUTURE WORK

This project shows that tree-based models, tested fairly on unseen dates, can forecast parking occupancy accurately one hour ahead (MAE 0.0217, 60.6% better than the best simple rule) and somewhat better than a strong simple rule for the next day (MAE 0.0479, 11.3% better). SHAP shows the models rely on sensible inputs, and the error analysis shows where they struggle: weekends, nearly full car parks and unusual days. The web app turns these forecasts into parking recommendations and an admin view. Future work: collect real campus data with timetable and event information, including two-wheeler parking zones, which are common on Indian campuses and can be handled by the same pipeline; calibrate the likely ranges, test over more time periods, and add camera-based space detection.

ACKNOWLEDGMENT

The author chose this project, directed the work, set up and ran the system locally, published the code on GitHub and deployed the app on Streamlit Community Cloud, and reviewed and approved the results and this report. Claude (Anthropic), an AI assistant, was used to help write the code, run the analysis and draft the text.

REFERENCES

[1] F. Caicedo, C. Blazquez, and P. Miranda, "Prediction of parking space availability in real time," *Expert Syst. Appl.*, vol. 39, no. 8, 2012.

[2] T. Rajabioun and P. A. Ioannou, "On-street and off-street parking availability prediction using multivariate spatiotemporal models," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 5, pp. 2913–2924, 2015.

[3] E. I. Vlahogianni, K. Kepaptsoglou, V. Tsetsos, and M. G. Karlaftis, "A real-time parking prediction system for smart cities," *J. Intell. Transp. Syst.*, vol. 20, no. 2, pp. 192–204, 2016.

[4] D. H. Stolfi, E. Alba, and X. Yao, "Predicting car park occupancy rates in smart cities," in *Smart Cities (Smart-CT 2017)*, LNCS, vol. 10268. Cham: Springer, 2017, pp. 107–117, doi: 10.1007/978-3-319-59513-9_11.

[5] D. H. Stolfi, E. Alba, and X. Yao, "Can I park in the city center? Predicting car park occupancy rates in smart cities," *J. Urban Technol.*, vol. 27, no. 4, pp. 27–41, 2020, doi: 10.1080/10630732.2019.1586223.

[6] A. Camero, J. Toutouh, D. H. Stolfi, and E. Alba, "Evolutionary deep learning for car park occupancy prediction in smart cities," in *Learning and Intelligent Optimization (LION 12)*, LNCS, vol. 11353. Cham: Springer, 2019, doi: 10.1007/978-3-030-05348-2_32.

[7] X. Xiao *et al.*, "Parking prediction in smart cities: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 10, 2023.

[8] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.

[10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst. 30*, 2017, pp. 4765–4774.

[11] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nat. Mach. Intell.*, vol. 2, no. 1, pp. 56–67, 2020.

[12] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[13] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[14] D. Stolfi, "Parking Birmingham," UCI Machine Learning Repository, 2017, doi: 10.24432/C51K5Z.

[15] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Min.*, 2008, pp. 413–422.

[16] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241–259, 1992.

[17] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.